

ARTDCMEM-2418 · MEM_43_MRAM · S32K566

Code MRAM data sections and the OTA A/B swap

What the driver does, line by line, when Fee writes and reads 10 KB of data in the Code MRAM before and after the swap

1. The configuration and where the data is

`MemDataInMRamCodeEnabled` = TRUE and `MemDataInMRamCodeSize` = 0x2800 (10 KB). The driver then treats the last 10 KB of each OTA half of the Code MRAM as data. In the Full swap:

Section	Addresses	Pair	Used here by
End of the active half	0x08FFD800 – 0x08FFFFFF	3	Fee Cluster0 (MemSectorBatch FeeCluster0)
End of the passive half	0x09FFD800 – 0x09FFFFFF	7	Fee Cluster1 (MemSectorBatch FeeCluster1)
Rest of each half	0x08000000 – 0x08FFD7FF, 0x09000000 – 0x09FFD7FF	0–3, 4–7	the application images (updater)

The swap does not move any data: the MRAMC only changes which physical pair of 4 MB sits behind each logical pair (Full swap: pair $k \leftrightarrow$ pair $k+4$). The CPU, and the driver, always work with logical addresses.

No swap (OTA_SEL = 0)

logical address (what the CPU / driver uses) → physical pair

active half	0x08000000	logical pair 0 → physical pair 0
	0x08400000	logical pair 1 → physical pair 1
	0x08800000	logical pair 2 → physical pair 2
	0x08C00000	logical pair 3 → physical pair 3
passive half	0x08FFD800	last 10 KB: Cluster0 data (physical pair 3)
	0x09000000	logical pair 4 → physical pair 4
	0x09400000	logical pair 5 → physical pair 5
	0x09800000	logical pair 6 → physical pair 6
	0x09C00000	logical pair 7 → physical pair 7
	0x09FFD800	last 10 KB: Cluster1 data (physical pair 7)

Full swap (OTA_SEL = 1)

logical address (what the CPU / driver uses) → physical pair

active half	0x08000000	logical pair 0 → physical pair 4
	0x08400000	logical pair 1 → physical pair 5
	0x08800000	logical pair 2 → physical pair 6
	0x08C00000	logical pair 3 → physical pair 7
passive half	0x08FFD800	last 10 KB: Cluster1 data (physical pair 7)
	0x09000000	logical pair 4 → physical pair 0
	0x09400000	logical pair 5 → physical pair 1
	0x09800000	logical pair 6 → physical pair 2
	0x09C00000	logical pair 7 → physical pair 3
	0x09FFD800	last 10 KB: Cluster0 data (physical pair 3)

Colour of a pair = the physical pair behind it (blue: physical pairs 0–3, orange: 4–7). The 10 KB sections are drawn much larger than they are.

After the swap Fee still asks for 0x08FFD800 (Cluster0), but there the CPU now sees physical pair 7 (Cluster1 data): the driver therefore accesses 0x09FFD800.

The rule the driver applies

While a swap is active (value latched at init), an address in the section of the active half is moved one half higher and an address in the section of the passive half one half lower (16 MB in the Full swap, 12 MB in the Partial swap). Every other address is accessed as requested. So Fee always reaches the same physical memory, and the updater still writes the passive image where it asks.

2. The code that decides the address

The OTA configuration is read once, in `MRam_Ip_Init`, and kept in `MRam_Ip_OtaNumFixed` (line 254, "no swap" until init). Read, Write, Erase and their AXI variants pass the requested address through `MRAM_IP_REBASED_ADDRESS`; the lock APIs do not, because they already receive the moved address from `MRam_Ip_ProcessOperation`.

`MRam_Ip.c` lines 252–254

the latched OTA configuration

```

252  /*! @brief OTA configuration (GPR_RWD_0 OTA_NUM_FIXED_LPART, see MRam_Ip_GetOtaNumFixed) latched by
253  *      MRam_Ip_Init. The swap only changes at reset; before MRam_Ip_Init nothing is swapped. */
254  static uint32 MRam_Ip_OtaNumFixed = MRAM_IP_OTA_NUM_FIXED_ALL;
```

MRam_Ip.c lines 5489–5490

in MRam_Ip_Init

```
5489  /* Latch the OTA configuration once: the swap only changes at reset. */
5490  MRam_Ip_OtaNumFixed = MRam_Ip_GetOtaNumFixed();
```

MRam_Ip.c lines 2021–2046

MRam_Ip_GetOtaNumFixed

```
2021  /**
2022  * @brief Read the OTA configuration (GPR_RWD_0 OTA_NUM_FIXED_LPART), latched by MRam_Ip_Init.
2023  * @details Nothing is swapped without OTA_SEL, nor with OTA_SEL and OTA_NUM_FIXED_LPART = 0x10 (All
2024  *         fixed): both are reported as MRAM_IP_OTA_NUM_FIXED_ALL.
2025  *
2026  * @return uint32 - OTA_NUM_FIXED_LPART, or MRAM_IP_OTA_NUM_FIXED_ALL when OTA_SEL is 0.
2027  */
2028  static uint32 MRam_Ip_GetOtaNumFixed(void)
2029  {
2030      /* Default: OTA_SEL = 0, nothing is swapped - reported as "All fixed - no OTA". */
2031      uint32 otaNumFixed = MRAM_IP_OTA_NUM_FIXED_ALL;
2032      /* Single volatile read of GPR_RWD_0 (MISRA C:2012 Rule 13.2), which holds both OTA_SEL and
2033       * OTA_NUM_FIXED_LPART. GPR_RWD_0 is write-locked by the bootloader (LOCK_EN0/LOCK_EN1 set, see
2034       * S32K5xx Reference Manual, GPR1 chapter), so the value is stable at runtime. */
2035      uint32 gprVal = IP_GPR_1->GPR_RWD_0;
2036
2037      if ((gprVal & K566_GPR1_GPR_RWD_0_OTA_SEL_MASK) != 0U)
2038      {
2039          /* OTA_SEL set: the configuration comes from OTA_NUM_FIXED_LPART (bits 11-6). The swap only
2040           * takes place for 0x00 (Full) and 0x04 (Partial); 0x10 (All fixed) means no swap. */
2041          otaNumFixed = (gprVal & K566_GPR1_GPR_RWD_0_OTA_NUM_FIXED_LPART_MASK)
2042                      >> K566_GPR1_GPR_RWD_0_OTA_NUM_FIXED_LPART_SHIFT;
2043      }
2044
2045      return otaNumFixed;
2046  }
```

How to read the next pages

"Path through the code" lists the functions called, with their arguments and return values. The tables list every executed line of the functions that matter; the "Result" column shows the value produced by the line, or both sides of each comparison of an `if`. The values come from running the driver sources (Mem_43_MRAM.c, Mem_43_MRAM_Ipw.c, MRam_Ip.c) on a host MRAM model that applies the OTA remap; line numbers refer to MRam_Ip.c.

MRam_Ip.c lines 2048–2104

MRam_Ip_GetRebasedAddress

```

2048 #if (STD_ON == MRAM_IP_DATA_IN_CODE_MRAM)
2049 /**
2050  * @brief Return the address at which the data of a Code MRAM data section address is found.
2051  * @details The data sections are the last MRAM_IP_DATA_IN_CODE_MRAM_SIZE_U32 bytes of the active and
2052  * of
2053  * the passive half. While a swap is active, the data of the active half section is found one
2054  * half
2055  * higher and the data of the passive half section one half lower (16 MB in the Full swap, 12
2056  * MB
2057  * in the Partial swap). Otherwise the address is returned unchanged.
2058  *
2059  * @param[in] Address Address requested by the caller.
2060  * @return MRam_Ip_AddressType - the address to access.
2061  */
2062 static MRam_Ip_AddressType MRam_Ip_GetRebasedAddress(MRam_Ip_AddressType Address)
2063 {
2064     /* By default the address is accessed as requested. */
2065     MRam_Ip_AddressType otaAddress = Address;
2066     /* Size of one swapped half, 0 while nothing is swapped. */
2067     uint32 halfSize = 0U;
2068
2069     /* OTA configuration latched by MRam_Ip_Init (the swap only changes at reset). */
2070     if (MRAM_IP_OTA_NUM_FIXED_FULL == MRam_Ip_OtaNumFixed)
2071     {
2072         /* Full swap active: halves of 16 MB. */
2073         halfSize = MRAM_IP_OTA_FULL_SWAP_OFFSET;
2074     }
2075     else if (MRAM_IP_OTA_NUM_FIXED_PARTIAL == MRam_Ip_OtaNumFixed)
2076     {
2077         /* Partial swap active: halves of 12 MB above the fixed pairs 0..1. */
2078         halfSize = MRAM_IP_OTA_PARTIAL_SWAP_OFFSET;
2079     }
2080     else
2081     {
2082         /* No swap or undefined encoding: nothing is moved. */
2083     }
2084
2085     if (0U != halfSize)
2086     {
2087         if ((Address >= (MRAM_END_ADDRESS_CODE - MRAM_IP_DATA_IN_CODE_MRAM_SIZE_U32)) &&
2088             (Address < MRAM_END_ADDRESS_CODE))
2089         {
2090             /* Section at the end of the passive half (end of the Code MRAM): its data is now seen
2091              * one half lower. */
2092             otaAddress = Address - halfSize;
2093         }
2094         else if ((Address >= (MRAM_END_ADDRESS_CODE - halfSize - MRAM_IP_DATA_IN_CODE_MRAM_SIZE_U32))
2095             &&
2096             (Address < (MRAM_END_ADDRESS_CODE - halfSize)))
2097         {
2098             /* Section at the end of the active half: its data is now seen one half higher. */
2099             otaAddress = Address + halfSize;
2100         }
2101         else
2102         {
2103             /* Outside the data sections: accessed as requested. */
2104         }
2105     }
2106
2107     return otaAddress;
2108 }

```

3. Full swap: step by step

B1 Boot with OTA_SEL = 0: Mem_43_MRAM_Init -> MRam_Ip_Init latches the OTA configuration

State of the device

GPR_RWD_0 = 0x00000000 (OTA_SEL = 0, OTA_NUM_FIXED_LPART = 0x00): **no swap**. Logical pair → physical pair: 0→0, 1→1, 2→2, 3→3, 4→4, 5→5, 6→6, 7→7.

At start-up Fee initialises the Mem driver, which initialises MRam_Ip. MRam_Ip_Init reads GPR_RWD_0 once and keeps the result: here nothing is swapped.

Path through the code

```
Mem_43_MRAM_Init() Mem_43_MRAM.c:1286
  Mem_43_MRAM_IPW_Init() → 0 Mem_43_MRAM_Ipw.c:291
    MRam_Ip_Init() MRam_Ip.c:5474
      MRam_Ip_GetOtaNumFixed() → 0x10 MRam_Ip.c:2028
```

MRam_Ip_GetOtaNumFixed reads GPR_RWD_0 once, at init

Line	Code executed	Result (values from the run)
2031	uint32 otaNumFixed = MRAM_IP_OTA_NUM_FIXED_ALL;	otaNumFixed = 0x10
2035	uint32 gprVal = IP_GPR_1->GPR_RWD_0;	gprVal = 0
2037	if ((gprVal & K566_GPR1_GPR_RWD_0_OTA_SEL_MASK) != 0U)	0 != 0 → false
2045	return otaNumFixed;	returns 0x10

Result

MRam_Ip_Init (line 5490) keeps the returned value: MRam_Ip_OtaNumFixed = 0x10 (no swap). Every Read / Write / Erase until the next reset uses this value; GPR_RWD_0 is not read again.

W1 Before the swap: Fee writes 32 bytes (A0..BF) to Cluster0 at 0x08FFD800

State of the device

GPR_RWD_0 = 0x00000000 (OTA_SEL = 0, OTA_NUM_FIXED_LPART = 0x00): **no swap**. Logical pair → physical pair: 0→0, 1→1, 2→2, 3→3, 4→4, 5→5, 6→6, 7→7.

Fee writes a first record (32 bytes, A0..BF) at the start of Cluster0. Nothing is swapped, so the address is not changed and the page is programmed in physical pair 3.

Path through the code

```
Mem_43_MRAM_Write(TargetAddress=0x08FFD800, Length=0x20) → 0 Mem_43_MRAM.c:1621
  Mem_43_MRAM_GetSectorBatch(Address=0x08FFD800) Mem_43_MRAM.c:443 // MemSectorBatch FeeCluster0 [0x08FFD800,
  0x08FFFFFF]
  Mem_43_MRAM_MainFunction() Mem_43_MRAM.c:2058
    Mem_43_MRAM_IPW_Write() → MEM_43_MRAM_JOB_OK Mem_43_MRAM_Ipw.c:844
      MRam_Ip_Write(DestAddress=0x08FFD800, SizeInBytes=0x20) → MRAM_IP_STATUS_SUCCESS MRam_Ip.c:7348
        MRam_Ip_GetRebasedAddress(Address=0x08FFD800) → 0x08FFD800 MRam_Ip.c:2059
          MRam_Ip_ProcessOperation(DestAddress=0x08FFD800, SizeInBytes=0x20) → MRAM_IP_STATUS_SUCCESS MRam_Ip.c:7033
            MRam_Ip_ValidateOperationParameters(MemAddress=0x08FFD800, SizeInBytes=0x20) → MRAM_IP_STATUS_SUCCESS
MRam_Ip.c:436
```

```

MRam_Ip_ConfigureLocksInRange(StartAddress=0x08FFD800, SizeInBytes=0x20, SetLock=0) →
MRAM_IP_STATUS_SUCCESS MRam_Ip.c:3439
MRam_Ip_GetPartition(Address=0x08FFD800) → MRAM_IP_PARTITION_LARGE_3_LOWER MRam_Ip.c:2001
MRam_Ip_ClearLock(Address=0x08FFD800) → MRAM_IP_STATUS_SUCCESS MRam_Ip.c:4432
MRam_Ip_GetLock(Address=0x08FFD800) → MRAM_IP_STATUS_PARTITION_UNLOCKED MRam_Ip.c:4302
MRam_Ip_GetPartition(Address=0x08FFD800) → MRAM_IP_PARTITION_LARGE_3_LOWER MRam_Ip.c:2001
MRam_Ip_IsPartitionLocked(Partition=MRAM_IP_PARTITION_LARGE_3_LOWER) → 0 MRam_Ip.c:2386
MRam_Ip_CheckMramcLock(Partition=MRAM_IP_PARTITION_LARGE_3_LOWER) → 0 MRam_Ip.c:2333
MRam_Ip_CheckF16asmLockLarge(Partition=MRAM_IP_PARTITION_LARGE_3_LOWER, f16asmLock0=0,
otaNumFixed=0x10) → 0 MRam_Ip.c:2222
MRam_Ip_CheckLargeLockNoSwap(Partition=MRAM_IP_PARTITION_LARGE_3_LOWER, f16asmLock0=0)
→ 0 MRam_Ip.c:2117
MRam_Ip_WriteOp_PageLoop() MRam_Ip.c:6539
MRam_Ip_AcquireWriteSemaphore(MRamAddress=0x08FFD800) → MRAM_IP_STATUS_SUCCESS MRam_Ip.c:1677
MRam_Ip_ExecuteSinglePage(Address=0x08FFD800, WriteSize=0x20) MRam_Ip.c:685
< MRAMC programs WADDR = 0x08FFD800 (logical) → logical pair 3 → physical pair 3, lower bank
MRam_Ip_WriteOp_Teardown() MRam_Ip.c:6762
MRam_Ip_ConfigureLocksInRange(StartAddress=0x08FFD800, SizeInBytes=0x20, SetLock=1) →
MRAM_IP_STATUS_SUCCESS MRam_Ip.c:3439
MRam_Ip_GetPartition(Address=0x08FFD800) → MRAM_IP_PARTITION_LARGE_3_LOWER MRam_Ip.c:2001
MRam_Ip_SetPartitionLock(Partition=MRAM_IP_PARTITION_LARGE_3_LOWER) MRam_Ip.c:2815
MRam_Ip_CheckMramcLock(Partition=MRAM_IP_PARTITION_LARGE_3_LOWER) → 1 MRam_Ip.c:2333

```

MRam_Ip_GetRebasedAddress which address is really accessed

Line	Code executed	Result (values from the run)
2062	MRam_Ip_AddressType otaAddress = Address;	otaAddress = 0x08FFD800
2064	uint32 halfSize = 0U;	halfSize = 0
2067	if (MRAM_IP_OTA_NUM_FIXED_FULL == MRam_Ip_OtaNumFixed)	0 == 0x10 → false
2072	else if (MRAM_IP_OTA_NUM_FIXED_PARTIAL == MRam_Ip_OtaNumFixed)	4 == 0x10 → false
2082	if (0U != halfSize)	0 != 0 → false
2103	return otaAddress;	returns 0x08FFD800

Result

The MRAMC programs the page at WADDR = 0x08FFD800 (logical). Logical pair 3 is backed by **physical pair 3** (lower bank).

Physical array, pair 3 offset 0x3FD800: A0 A1 A2 A3 A4 A5 A6 A7 A8 A9 AA AB AC AD AE AF B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 BA BB BC BD BE BF

W2 Before the swap: Fee writes 32 bytes (B0..CF) to Cluster1 at 0x09FFD800

State of the device

GPR_RWD_0 = 0x00000000 (OTA_SEL = 0, OTA_NUM_FIXED_LPART = 0x00): **no swap**. Logical pair → physical pair: 0→0, 1→1, 2→2, 3→3, 4→4, 5→5, 6→6, 7→7.

Fee writes a record (B0..CF) at the start of Cluster1, in the section of the passive half. Still no swap: physical pair 7.

Path through the code (main calls)

```

Mem_43_MRAM_Write(TargetAddress=0x09FFD800, Length=0x20) → 0 Mem_43_MRAM.c:1621
Mem_43_MRAM_GetSectorBatch(Address=0x09FFD800) Mem_43_MRAM.c:443 // MemSectorBatch FeeCluster1 [0x09FFD800,
0x09FFFFFF]
MRam_Ip_Write(DestAddress=0x09FFD800, SizeInBytes=0x20) → MRAM_IP_STATUS_SUCCESS MRam_Ip.c:7348
MRam_Ip_GetRebasedAddress(Address=0x09FFD800) → 0x09FFD800 MRam_Ip.c:2059
MRam_Ip_ProcessOperation(DestAddress=0x09FFD800, SizeInBytes=0x20) → MRAM_IP_STATUS_SUCCESS MRam_Ip.c:7033
MRam_Ip_ConfigureLocksInRange(StartAddress=0x09FFD800, SizeInBytes=0x20, SetLock=0) → MRAM_IP_STATUS_SUCCESS
MRam_Ip.c:3439
MRam_Ip_ClearLock(Address=0x09FFD800) → MRAM_IP_STATUS_SUCCESS MRam_Ip.c:4432
MRam_Ip_GetLock(Address=0x09FFD800) → MRAM_IP_STATUS_PARTITION_UNLOCKED MRam_Ip.c:4302

```

```
MRam_Ip_ExecuteSinglePage(Address=0x09FFD800, WriteSize=0x20) MRam_Ip.c:685
  < MRAMC programs WADDR = 0x09FFD800 (logical) → logical pair 7 → physical pair 7, lower bank
MRam_Ip_ConfigureLocksInRange(StartAddress=0x09FFD800, SizeInBytes=0x20, SetLock=1) → MRAM_IP_STATUS_SUCCESS
MRam_Ip.c:3439
  MRam_Ip_SetPartitionLock(Partition=MRAM_IP_PARTITION_LARGE_7_LOWER) MRam_Ip.c:2815
```

MRam_Ip_GetRebasedAddress which address is really accessed

Line	Code executed	Result (values from the run)
2062	MRam_Ip_AddressType otaAddress = Address;	otaAddress = 0x09FFD800
2064	uint32 halfSize = 0U;	halfSize = 0
2067	if (MRAM_IP_OTA_NUM_FIXED_FULL == MRam_Ip_OtaNumFixed)	0 == 0x10 → false
2072	else if (MRAM_IP_OTA_NUM_FIXED_PARTIAL == MRam_Ip_OtaNumFixed)	4 == 0x10 → false
2082	if (0U != halfSize)	0 != 0 → false
2103	return otaAddress;	returns 0x09FFD800

Result

The MRAMC programs the page at WADDR = 0x09FFD800 (logical). Logical pair 7 is backed by **physical pair 7** (lower bank).

Physical array, pair 7 offset 0x3FD800: B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 BA BB BC BD BE BF C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF

R1 Before the swap: Fee reads 32 bytes from Cluster0 at 0x08FFD800

State of the device

GPR_RWD_0 = 0x00000000 (OTA_SEL = 0, OTA_NUM_FIXED_LPART = 0x00): **no swap**. Logical pair → physical pair: 0→0, 1→1, 2→2, 3→3, 4→4, 5→5, 6→6, 7→7.

Fee reads the record back from Cluster0. No swap: the CPU reads physical pair 3 and gets A0..BF.

Path through the code

```
Mem_43_MRAM_Read(SourceAddress=0x08FFD800, Length=0x20) → 0 Mem_43_MRAM.c:1555
  Mem_43_MRAM_GetSectorBatch(Address=0x08FFD800) Mem_43_MRAM.c:443 // MemSectorBatch FeeCluster0 [0x08FFD800, 0x08FFFFFF]
  Mem_43_MRAM_MainFunction() Mem_43_MRAM.c:2058
    Mem_43_MRAM_IPW_Read() → MEM_43_MRAM_JOB_OK Mem_43_MRAM_Ipw.c:363
      MRam_Ip_Read(SrcAddress=0x08FFD800, SizeInBytes=0x20) → MRAM_IP_STATUS_SUCCESS MRam_Ip.c:7511
        MRam_Ip_GetRebasedAddress(Address=0x08FFD800) → 0x08FFD800 MRam_Ip.c:2059
          MRam_Ip_ValidateOperationParameters(MemAddress=0x08FFD800, SizeInBytes=0x20) → MRAM_IP_STATUS_SUCCESS
MRam_Ip.c:436
  MRam_Ip_ReadData(ReadAddress=0x08FFD800, SizeLeft=0x20) → 4 MRam_Ip.c:1872
    ... loop: MRam_Ip_ReadData 0x08FFD804, 0x08FFD808, 0x08FFD80C, 0x08FFD810, 0x08FFD814, 0x08FFD818, 0x08FFD81C (4 bytes each)
```

MRam_Ip_Read first loop iteration in full, the others summarised

Line	Code executed	Result (values from the run)
7523	MRam_Ip_LengthType destOffset = 0U;	destOffset = 0
7524	currentReadAddress = MRAM_IP_REBASED_ADDRESS(SrcAddress);	currentReadAddress = 0x08FFD800
	↳ MRam_Ip_GetRebasedAddress(Address=0x08FFD800) → returns 0x08FFD800	
7525	remainingBytesToRead = SizeInBytes;	remainingBytesToRead = 0x20
7527-7531	operationStatus = MRam_Ip_ValidateOperationParameters(currentReadAddress, DestAddressPtr, SizeInBytes, InstanceId, TRUE);	operationStatus = MRAM_IP_STATUS_SUCCESS

	↳ MRam_Ip_ValidateOperationParameters (MemAddress=0x08FFD800, SizeInBytes=0x20) → returns MRAM_IP_STATUS_SUCCESS	
7538	if (operationStatus == MRAM_IP_STATUS_SUCCESS)	MRAM_IP_STATUS_SUCCESS == MRAM_IP_STATUS_SUCCESS → true
7540	(void)Cache_Ip_InvalidateByAddr(CACHE_IP_CORE, CACHE_IP_DATA, currentReadAddress, SizeInBytes);	
	↳ cache invalidated for 0x08FFD800, 32 bytes (logical pair 3)	
7544	memoryType = MRam_Ip_GetAddressType (currentReadAddress);	memoryType = MRAM_IP_ADDRESS_AREA_TYPE_CODE
7547	isOtpMemoryRegion = (MRAM_IP_ADDRESS_AREA_TYPE_NS_OTP == memoryType);	isOtpMemoryRegion = 0
7549	if (operationStatus == MRAM_IP_STATUS_SUCCESS)	MRAM_IP_STATUS_SUCCESS == MRAM_IP_STATUS_SUCCESS → true
7552	if (isOtpMemoryRegion)	0 → false
7562	operationStatus = MRam_Ip_Read_CheckPortBuffersEnabled (memoryType, InstanceId);	operationStatus = MRAM_IP_STATUS_SUCCESS
7566	MRam_Ip_StatusType ClearStatus = MRam_Ip_GetMemoryErrorStatus ();	ClearStatus = MRAM_IP_STATUS_SUCCESS
7574	ClearStatus = MRam_Ip_DecodeAndClearErrstat0 (InstanceId, MRam_Ip_Errstat0AllKnownErrors);	ClearStatus = MRAM_IP_STATUS_SUCCESS
7584	if (operationStatus == MRAM_IP_STATUS_SUCCESS)	MRAM_IP_STATUS_SUCCESS == MRAM_IP_STATUS_SUCCESS → true
7586	MRam_Ip_Read_ConfigurePrefetch (memoryType, isOtpMemoryRegion, InstanceId);	
7590	while ((operationStatus == MRAM_IP_STATUS_SUCCESS) && (remainingBytesToRead > 0UL))	MRAM_IP_STATUS_SUCCESS == MRAM_IP_STATUS_SUCCESS → true and 0x20 > 0 → true
7597	operationStatus = MRam_Ip_Read_InvalidatePorts (InstanceId);	operationStatus = MRAM_IP_STATUS_SUCCESS
	↳ MRam_Ip_Read_InvalidatePorts () → returns MRAM_IP_STATUS_SUCCESS	
7599	if (operationStatus == MRAM_IP_STATUS_SUCCESS)	MRAM_IP_STATUS_SUCCESS == MRAM_IP_STATUS_SUCCESS → true
7602	bytesReadInOperation = MRam_Ip_ReadData (currentReadAddress, &DestAddressPtr[destOffset], remainingBytesToRead);	bytesReadInOperation = 4
	↳ MRam_Ip_ReadData (ReadAddress=0x08FFD800, SizeLeft=0x20) → returns 4	
7607	operationStatus = MRam_Ip_DecodeAndClearErrstat0 (InstanceId, MRam_Ip_Errstat0ReadDecodeMask);	operationStatus = MRAM_IP_STATUS_SUCCESS
7609	if (operationStatus == MRAM_IP_STATUS_SUCCESS)	MRAM_IP_STATUS_SUCCESS == MRAM_IP_STATUS_SUCCESS → true
7612	currentReadAddress += bytesReadInOperation;	currentReadAddress = 0x08FFD804
7613	destOffset += bytesReadInOperation;	destOffset = 4
7614	remainingBytesToRead -= bytesReadInOperation;	remainingBytesToRead = 0x1C
...	the loop body runs 7 more times, the same lines: MRam_Ip_ReadData (ReadAddress = 0x08FFD804, 0x08FFD808, 0x08FFD80C, 0x08FFD810, 0x08FFD814, 0x08FFD818, 0x08FFD81C), 4 bytes each; then the while condition is false (remainingBytesToRead = 0)	
7619	return operationStatus;	returns MRAM_IP_STATUS_SUCCESS

MRam_Ip_GetRebasedAddress which address is really accessed		
Line	Code executed	Result (values from the run)
2062	MRam_Ip_AddressType otaAddress = Address;	otaAddress = 0x08FFD800
2064	uint32 halfSize = 0U;	halfSize = 0
2067	if (MRAM_IP_OTA_NUM_FIXED_FULL == MRam_Ip_OtaNumFixed)	0 == 0x10 → false
2072	else if (MRAM_IP_OTA_NUM_FIXED_PARTIAL == MRam_Ip_OtaNumFixed)	4 == 0x10 → false
2082	if (0U != halfSize)	0 != 0 → false
2103	return otaAddress;	returns 0x08FFD800

Result

The CPU reads logical 0x08FFD800 ... → logical pair 3 → **physical pair 3**.

Data returned to the caller (0x08FFD800): A0 A1 A2 A3 A4 A5 A6 A7 A8 A9 AA AB AC AD AE AF B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 BA BB BC BD BE BF

B2 Boot after the swap (OTA_SEL = 1, Full swap): MRam_Ip_Init latches the OTA configuration

State of the device

GPR_RWD_0 = 0x00000020 (OTA_SEL = 1, OTA_NUM_FIXED_LPART = 0x00): **Full swap**. Logical pair → physical pair: 0→4, 1→5, 2→6, 3→7, 4→0, 5→1, 6→2, 7→3.

The updater has written the new image in the passive half and the device resets with the swap selected. MRam_Ip_Init latches the new configuration: Full swap. From now on logical pair 3 is physical pair 7 and logical pair 7 is physical pair 3.

Path through the code

```
Mem_43_MRAM_Init() Mem_43_MRAM.c:1286
  Mem_43_MRAM_IPW_Init() → 0 Mem_43_MRAM_Ipw.c:291
    MRam_Ip_Init() MRam_Ip.c:5474
      MRam_Ip_GetOtaNumFixed() → 0 MRam_Ip.c:2028
```

MRam_Ip_GetOtaNumFixed reads GPR_RWD_0 once, at init		
Line	Code executed	Result (values from the run)
2031	uint32 otaNumFixed = MRAM_IP_OTA_NUM_FIXED_ALL;	otaNumFixed = 0x10
2035	uint32 gprVal = IP_GPR_1->GPR_RWD_0;	gprVal = 0x20
2037	if ((gprVal & K566_GPR1_GPR_RWD_0_OTA_SEL_MASK) != 0U)	0x20 != 0 → true
2041–2042	otaNumFixed = (gprVal & K566_GPR1_GPR_RWD_0_OTA_NUM_FIXED_LPART_MASK) >> K566_GPR1_GPR_RWD_0_OTA_NUM_FIXED_LPART_SHIFT;	otaNumFixed = 0
2045	return otaNumFixed;	returns 0

Result

MRam_Ip_Init (line 5490) keeps the returned value: MRam_Ip_OtaNumFixed = 0x00 (Full swap). Every Read / Write / Erase until the next reset uses this value; GPR_RWD_0 is not read again.

R2 After the swap: Fee reads 32 bytes from Cluster0 at 0x08FFD800

State of the device

GPR_RWD_0 = 0x00000020 (OTA_SEL = 1, OTA_NUM_FIXED_LPART = 0x00): **Full swap**. Logical pair → physical pair: 0→4, 1→5, 2→6, 3→7, 4→0, 5→1, 6→2, 7→3.

Fee reads Cluster0 again at the same address, 0x08FFD800. Without the driver change it would now read physical pair 7 (Cluster1 data). The driver moves the address one half higher, to 0x09FFD800, which is physical pair 3: Fee gets its A0..BF back.

Path through the code

```
Mem_43_MRAM_Read(SourceAddress=0x08FFD800, Length=0x20) → 0 Mem_43_MRAM.c:1555
Mem_43_MRAM_GetSectorBatch(Address=0x08FFD800) Mem_43_MRAM.c:443 // MemSectorBatch FeeCluster0 [0x08FFD800,
0x08FFFFFF]
Mem_43_MRAM_MainFunction() Mem_43_MRAM.c:2058
Mem_43_MRAM_IPW_Read() → MEM_43_MRAM_JOB_OK Mem_43_MRAM_Ipw.c:363
MRam_Ip_Read(SrcAddress=0x08FFD800, SizeInBytes=0x20) → MRAM_IP_STATUS_SUCCESS MRam_Ip.c:7511
MRam_Ip_GetRebasedAddress(Address=0x08FFD800) → 0x09FFD800 MRam_Ip.c:2059
MRam_Ip_ValidateOperationParameters(MemAddress=0x09FFD800, SizeInBytes=0x20) → MRAM_IP_STATUS_SUCCESS
MRam_Ip.c:436
MRam_Ip_ReadData(ReadAddress=0x09FFD800, SizeLeft=0x20) → 4 MRam_Ip.c:1872
... loop: MRam_Ip_ReadData 0x09FFD804, 0x09FFD808, 0x09FFD80C, 0x09FFD810, 0x09FFD814, 0x09FFD818,
0x09FFD81C (4 bytes each)
```

MRam_Ip_Read first loop iteration in full, the others summarised

Line	Code executed	Result (values from the run)
7523	MRam_Ip_LengthType destOffset = 0U;	destOffset = 0
7524	currentReadAddress = MRAM_IP_REBASED_ADDRESS(SrcAddress);	currentReadAddress = 0x09FFD800
	↳ MRam_Ip_GetRebasedAddress(Address=0x08FFD800) → returns 0x09FFD800	
7525	remainingBytesToRead = SizeInBytes;	remainingBytesToRead = 0x20
7527–7531	operationStatus = MRam_Ip_ValidateOperationParameters(currentReadAddress, DestAddressPtr, SizeInBytes, InstanceId, TRUE);	operationStatus = MRAM_IP_STATUS_SUCCESS
	↳ MRam_Ip_ValidateOperationParameters(MemAddress=0x09FFD800, SizeInBytes=0x20) → returns MRAM_IP_STATUS_SUCCESS	
7538	if (operationStatus == MRAM_IP_STATUS_SUCCESS)	MRAM_IP_STATUS_SUCCESS == MRAM_IP_STATUS_SUCCESS → true
7540	(void)Cache_Ip_InvalidateByAddr(CACHE_IP_CORE, CACHE_IP_DATA, currentReadAddress, SizeInBytes);	
	↳ cache invalidated for 0x09FFD800, 32 bytes (logical pair 7)	
7544	memoryType = MRam_Ip_GetAddressType(currentReadAddress);	memoryType = MRAM_IP_ADDRESS_AREA_TYPE_CODE
7547	isOtpMemoryRegion = (MRAM_IP_ADDRESS_AREA_TYPE_NS_OTP == memoryType);	isOtpMemoryRegion = 0
7549	if(operationStatus == MRAM_IP_STATUS_SUCCESS)	MRAM_IP_STATUS_SUCCESS == MRAM_IP_STATUS_SUCCESS → true
7552	if (isOtpMemoryRegion)	0 → false
7562	operationStatus = MRam_Ip_Read_CheckPortBuffersEnabled(memoryType, InstanceId);	operationStatus = MRAM_IP_STATUS_SUCCESS
7566	MRam_Ip_StatusType ClearStatus = MRam_Ip_GetMemoryErrorStatus();	ClearStatus = MRAM_IP_STATUS_SUCCESS
7574	ClearStatus = MRam_Ip_DecodeAndClearErrstat0(InstanceId, MRam_Ip_Errstat0AllKnownErrors);	ClearStatus = MRAM_IP_STATUS_SUCCESS
7584	if (operationStatus == MRAM_IP_STATUS_SUCCESS)	MRAM_IP_STATUS_SUCCESS == MRAM_IP_STATUS_SUCCESS → true
7586	MRam_Ip_Read_ConfigurePrefetch(memoryType, isOtpMemoryRegion, InstanceId);	

7590	<code>while ((operationStatus == MRAM_IP_STATUS_SUCCESS) && (remainingBytesToRead > 0UL))</code>	<code>MRAM_IP_STATUS_SUCCESS == MRAM_IP_STATUS_SUCCESS → true</code> and <code>0x20 > 0 → true</code>
7597	<code>operationStatus = MRam_Ip_Read_InvalidatePorts(InstanceId);</code>	<code>operationStatus = MRAM_IP_STATUS_SUCCESS</code>
	↳ <code>MRam_Ip_Read_InvalidatePorts()</code> → returns <code>MRAM_IP_STATUS_SUCCESS</code>	
7599	<code>if (operationStatus == MRAM_IP_STATUS_SUCCESS)</code>	<code>MRAM_IP_STATUS_SUCCESS == MRAM_IP_STATUS_SUCCESS → true</code>
7602	<code>bytesReadInOperation = MRam_Ip_ReadData(currentReadAddress, &DestAddressPtr[destOffset], remainingBytesToRead);</code>	<code>bytesReadInOperation = 4</code>
	↳ <code>MRam_Ip_ReadData(ReadAddress=0x09FFD800, SizeLeft=0x20) → returns 4</code>	
7607	<code>operationStatus = MRam_Ip_DecodeAndClearErrstat0(InstanceId, MRam_Ip_Errstat0ReadDecodeMask);</code>	<code>operationStatus = MRAM_IP_STATUS_SUCCESS</code>
7609	<code>if (operationStatus == MRAM_IP_STATUS_SUCCESS)</code>	<code>MRAM_IP_STATUS_SUCCESS == MRAM_IP_STATUS_SUCCESS → true</code>
7612	<code>currentReadAddress += bytesReadInOperation;</code>	<code>currentReadAddress = 0x09FFD804</code>
7613	<code>destOffset += bytesReadInOperation;</code>	<code>destOffset = 4</code>
7614	<code>remainingBytesToRead -= bytesReadInOperation;</code>	<code>remainingBytesToRead = 0x1C</code>
...	<i>the loop body runs 7 more times, the same lines: <code>MRam_Ip_ReadData(ReadAddress = 0x09FFD804, 0x09FFD808, 0x09FFD80C, 0x09FFD810, 0x09FFD814, 0x09FFD818, 0x09FFD81C)</code>, 4 bytes each; then the <code>while</code> condition is false (<code>remainingBytesToRead = 0</code>)</i>	
7619	<code>return operationStatus;</code>	returns <code>MRAM_IP_STATUS_SUCCESS</code>

MRam_Ip_GetRebasedAddress which address is really accessed

Line	Code executed	Result (values from the run)
2062	<code>MRam_Ip_AddressType otaAddress = Address;</code>	<code>otaAddress = 0x08FFD800</code>
2064	<code>uint32 halfSize = 0U;</code>	<code>halfSize = 0</code>
2067	<code>if (MRAM_IP_OTA_NUM_FIXED_FULL == MRam_Ip_OtaNumFixed)</code>	<code>0 == 0 → true</code>
2070	<code>halfSize = MRAM_IP_OTA_FULL_SWAP_OFFSET;</code>	<code>halfSize = 0x01000000</code>
2082	<code>if (0U != halfSize)</code>	<code>0 != 0x01000000 → true</code>
2084–2085	<code>if ((Address >= (MRAM_END_ADDRESS_CODE - MRAM_IP_DATA_IN_CODE_MRAM_SIZE_U32)) && (Address < MRAM_END_ADDRESS_CODE))</code>	<code>0x08FFD800 >= 0x09FFD800 → false</code> and <code>0x08FFD800 < 0x0A000000 → true</code>
2091–2092	<code>else if ((Address >= (MRAM_END_ADDRESS_CODE - halfSize - MRAM_IP_DATA_IN_CODE_MRAM_SIZE_U32)) && (Address < (MRAM_END_ADDRESS_CODE - halfSize)))</code>	<code>0x08FFD800 >= 0x08FFD800 → true</code> and <code>0x08FFD800 < 0x09000000 → true</code>
2095	<code>otaAddress = Address + halfSize;</code>	<code>otaAddress = 0x09FFD800</code>
2103	<code>return otaAddress;</code>	returns <code>0x09FFD800</code>

Result

The CPU reads logical 0x09FFD800 ... → logical pair 7 → **physical pair 3**.

Data returned to the caller (0x08FFD800): A0 A1 A2 A3 A4 A5 A6 A7 A8 A9 AA AB AC AD AE AF B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 BA BB BC BD BE BF

R3 After the swap: Fee reads 32 bytes from Cluster1 at 0x09FFD800

State of the device

GPR_RWD_0 = 0x00000020 (OTA_SEL = 1, OTA_NUM_FIXED_LPART = 0x00): **Full swap**. Logical pair → physical pair: 0→4, 1→5, 2→6, 3→7, 4→0, 5→1, 6→2, 7→3.

Fee reads Cluster1 at 0x09FFD800. The driver moves the address one half lower, to 0x08FFD800, which is now physical pair 7: Fee gets B0..CF back.

Path through the code (main calls)

```
Mem_43_MRAM_Read(SourceAddress=0x09FFD800, Length=0x20) → 0 Mem_43_MRAM.c:1555
  Mem_43_MRAM_GetSectorBatch(Address=0x09FFD800) Mem_43_MRAM.c:443 // MemSectorBatch FeeCluster1 [0x09FFD800,
  0x09FFFFFFF]
  MRam_Ip_Read(SrcAddress=0x09FFD800, SizeInBytes=0x20) → MRAM_IP_STATUS_SUCCESS MRam_Ip.c:7511
    MRam_Ip_GetRebasedAddress(Address=0x09FFD800) → 0x08FFD800 MRam_Ip.c:2059
    MRam_Ip_ReadData(ReadAddress=0x08FFD800, SizeLeft=0x20) → 4 MRam_Ip.c:1872
      ... loop: MRam_Ip_ReadData 0x08FFD804, 0x08FFD808, 0x08FFD80C, 0x08FFD810, 0x08FFD814, 0x08FFD818, 0x08FFD81C (4
      bytes each)
```

MRam_Ip_GetRebasedAddress which address is really accessed

Line	Code executed	Result (values from the run)
2062	MRam_Ip_AddressType otaAddress = Address;	otaAddress = 0x09FFD800
2064	uint32 halfSize = 0U;	halfSize = 0
2067	if (MRAM_IP_OTA_NUM_FIXED_FULL == MRam_Ip_OtaNumFixed)	0 == 0 → true
2070	halfSize = MRAM_IP_OTA_FULL_SWAP_OFFSET;	halfSize = 0x01000000
2082	if (0U != halfSize)	0 != 0x01000000 → true
2084–2085	if ((Address >= (MRAM_END_ADDRESS_CODE - MRAM_IP_DATA_IN_CODE_MRAM_SIZE_U32)) && (Address < MRAM_END_ADDRESS_CODE))	0x09FFD800 >= 0x09FFD800 → true and 0x09FFD800 < 0x0A000000 → true
2089	otaAddress = Address - halfSize;	otaAddress = 0x08FFD800
2103	return otaAddress;	returns 0x08FFD800

Result

The CPU reads logical 0x08FFD800 ... → logical pair 3 → physical pair 7.

Data returned to the caller (0x09FFD800): B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 BA BB BC BD BE BF C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF

W3 After the swap: Fee writes 32 bytes (C0..DF) to Cluster0 at 0x08FFD820

State of the device

GPR_RWD_0 = 0x00000020 (OTA_SEL = 1, OTA_NUM_FIXED_LPART = 0x00): Full swap. Logical pair → physical pair: 0→4, 1→5, 2→6, 3→7, 4→0, 5→1, 6→2, 7→3.

Fee writes a new record (C0..DF) in Cluster0 at 0x08FFD820. The driver writes it at 0x09FFD820, physical pair 3, next to the A0..BF record. The lock handling works on the moved address (logical pair 7), which is the pair that is really written.

Path through the code

```
Mem_43_MRAM_Write(TargetAddress=0x08FFD820, Length=0x20) → 0 Mem_43_MRAM.c:1621
  Mem_43_MRAM_GetSectorBatch(Address=0x08FFD820) Mem_43_MRAM.c:443 // MemSectorBatch FeeCluster0 [0x08FFD800,
  0x08FFFFFFF]
  Mem_43_MRAM_MainFunction() Mem_43_MRAM.c:2058
    Mem_43_MRAM_IPW_Write() → MEM_43_MRAM_JOB_OK Mem_43_MRAM_Ipw.c:844
      MRam_Ip_Write(DestAddress=0x08FFD820, SizeInBytes=0x20) → MRAM_IP_STATUS_SUCCESS MRam_Ip.c:7348
        MRam_Ip_GetRebasedAddress(Address=0x08FFD820) → 0x09FFD820 MRam_Ip.c:2059
        MRam_Ip_ProcessOperation(DestAddress=0x09FFD820, SizeInBytes=0x20) → MRAM_IP_STATUS_SUCCESS MRam_Ip.c:7033
        MRam_Ip_ValidateOperationParameters(MemAddress=0x09FFD820, SizeInBytes=0x20) → MRAM_IP_STATUS_SUCCESS
        MRam_Ip.c:436
        MRam_Ip_ConfigureLocksInRange(StartAddress=0x09FFD820, SizeInBytes=0x20, SetLock=0) →
        MRAM_IP_STATUS_SUCCESS MRam_Ip.c:3439
        MRam_Ip_GetPartition(Address=0x09FFD820) → MRAM_IP_PARTITION_LARGE_7_UPPER MRam_Ip.c:2001
```

```

MRam_Ip_ClearLock(Address=0x09FFD820) → MRAM_IP_STATUS_SUCCESS MRam_Ip.c:4432
MRam_Ip_GetLock(Address=0x09FFD820) → MRAM_IP_STATUS_PARTITION_UNLOCKED MRam_Ip.c:4302
MRam_Ip_GetPartition(Address=0x09FFD820) → MRAM_IP_PARTITION_LARGE_7_UPPER MRam_Ip.c:2001
MRam_Ip_IsPartitionLocked(Partition=MRAM_IP_PARTITION_LARGE_7_UPPER) → 0 MRam_Ip.c:2386
MRam_Ip_CheckMramcLock(Partition=MRAM_IP_PARTITION_LARGE_7_UPPER) → 0 MRam_Ip.c:2333
MRam_Ip_CheckF16asmLockLarge(Partition=MRAM_IP_PARTITION_LARGE_7_UPPER, f16asmLock0=0,
otaNumFixed=0) → 0 MRam_Ip.c:2222
MRam_Ip_CheckLargeLockSwap(Partition=MRAM_IP_PARTITION_LARGE_7_UPPER, f16asmLock0=0,
FixedPairs=0) → 0 MRam_Ip.c:2154
MRam_Ip_WriteOp_PageLoop() MRam_Ip.c:6539
MRam_Ip_AcquireWriteSemaphore(MRamAddress=0x09FFD820) → MRAM_IP_STATUS_SUCCESS MRam_Ip.c:1677
MRam_Ip_ExecuteSinglePage(Address=0x09FFD820, WriteSize=0x20) MRam_Ip.c:685
    < MRAMC programs WADDR = 0x09FFD820 (logical) → logical pair 7 → physical pair 3, upper bank
MRam_Ip_WriteOp_Teardown() MRam_Ip.c:6762
MRam_Ip_ConfigureLocksInRange(StartAddress=0x09FFD820, SizeInBytes=0x20, SetLock=1) →
MRAM_IP_STATUS_SUCCESS MRam_Ip.c:3439
MRam_Ip_GetPartition(Address=0x09FFD820) → MRAM_IP_PARTITION_LARGE_7_UPPER MRam_Ip.c:2001
MRam_Ip_SetPartitionLock(Partition=MRAM_IP_PARTITION_LARGE_7_UPPER) MRam_Ip.c:2815
MRam_Ip_CheckMramcLock(Partition=MRAM_IP_PARTITION_LARGE_7_UPPER) → 1 MRam_Ip.c:2333

```

MRam_Ip_GetRebasedAddress which address is really accessed

Line	Code executed	Result (values from the run)
2062	MRam_Ip_AddressType otaAddress = Address;	otaAddress = 0x08FFD820
2064	uint32 halfSize = 0U;	halfSize = 0
2067	if (MRAM_IP_OTA_NUM_FIXED_FULL == MRam_Ip_OtaNumFixed)	0 == 0 → true
2070	halfSize = MRAM_IP_OTA_FULL_SWAP_OFFSET;	halfSize = 0x01000000
2082	if (0U != halfSize)	0 != 0x01000000 → true
2084–2085	if ((Address >= (MRAM_END_ADDRESS_CODE - MRAM_IP_DATA_IN_CODE_MRAM_SIZE_U32)) && (Address < MRAM_END_ADDRESS_CODE))	0x08FFD820 >= 0x09FFD800 → false and 0x08FFD820 < 0x0A000000 → true
2091–2092	else if ((Address >= (MRAM_END_ADDRESS_CODE - halfSize - MRAM_IP_DATA_IN_CODE_MRAM_SIZE_U32)) && (Address < (MRAM_END_ADDRESS_CODE - halfSize)))	0x08FFD820 >= 0x08FFD800 → true and 0x08FFD820 < 0x09000000 → true
2095	otaAddress = Address + halfSize;	otaAddress = 0x09FFD820
2103	return otaAddress;	returns 0x09FFD820

Result

The MRAMC programs the page at WADDR = 0x09FFD820 (logical). Logical pair 7 is backed by **physical pair 3** (upper bank).

Physical array, pair 3 offset 0x3FD820: C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF D0 D1 D2 D3 D4 D5 D6 D7 D8 D9 DA DB DC DD DE DF

W4 After the swap: the updater writes 32 bytes of the next image at 0x09000000

State of the device

GPR_RWD_0 = 0x00000020 (OTA_SEL = 1, OTA_NUM_FIXED_LPART = 0x00): **Full swap**. Logical pair → physical pair: 0→4, 1→5, 2→6, 3→7, 4→0, 5→1, 6→2, 7→3.

The updater writes the next image in the passive half at 0x09000000. This address is outside the data sections: it is not moved and goes to physical pair 0, the passive image.

Path through the code (main calls)

```

Mem_43_MRAM_Write(TargetAddress=0x09000000, Length=0x20) → 0 Mem_43_MRAM.c:1621
Mem_43_MRAM_GetSectorBatch(Address=0x09000000) Mem_43_MRAM.c:443 // MemSectorBatch Image (passive half)
[0x09000000, 0x09FFD7FF]
MRam_Ip_Write(DestAddress=0x09000000, SizeInBytes=0x20) → MRAM_IP_STATUS_SUCCESS MRam_Ip.c:7348

```

```

MRam_Ip_GetRebasedAddress(Address=0x09000000) → 0x09000000 MRam_Ip.c:2059
MRam_Ip_ProcessOperation(DestAddress=0x09000000, SizeInBytes=0x20) → MRAM_IP_STATUS_SUCCESS MRam_Ip.c:7033
MRam_Ip_ConfigureLocksInRange(StartAddress=0x09000000, SizeInBytes=0x20, SetLock=0) → MRAM_IP_STATUS_SUCCESS
MRam_Ip.c:3439
MRam_Ip_ClearLock(Address=0x09000000) → MRAM_IP_STATUS_SUCCESS MRam_Ip.c:4432
MRam_Ip_GetLock(Address=0x09000000) → MRAM_IP_STATUS_PARTITION_UNLOCKED MRam_Ip.c:4302
MRam_Ip_ExecuteSinglePage(Address=0x09000000, WriteSize=0x20) MRam_Ip.c:685
✗ MRAMC programs WADDR = 0x09000000 (logical) → logical pair 4 → physical pair 0, lower bank
MRam_Ip_ConfigureLocksInRange(StartAddress=0x09000000, SizeInBytes=0x20, SetLock=1) → MRAM_IP_STATUS_SUCCESS
MRam_Ip.c:3439
MRam_Ip_SetPartitionLock(Partition=MRAM_IP_PARTITION_LARGE_4_LOWER) MRam_Ip.c:2815

```

MRam_Ip_GetRebasedAddress which address is really accessed

Line	Code executed	Result (values from the run)
2062	MRam_Ip_AddressType otaAddress = Address;	otaAddress = 0x09000000
2064	uint32 halfSize = 0U;	halfSize = 0
2067	if (MRAM_IP_OTA_NUM_FIXED_FULL == MRam_Ip_OtaNumFixed)	0 == 0 → true
2070	halfSize = MRAM_IP_OTA_FULL_SWAP_OFFSET;	halfSize = 0x01000000
2082	if (0U != halfSize)	0 != 0x01000000 → true
2084–2085	if ((Address >= (MRAM_END_ADDRESS_CODE - MRAM_IP_DATA_IN_CODE_MRAM_SIZE_U32)) && (Address < MRAM_END_ADDRESS_CODE))	0x09000000 >= 0x09FFD800 → false and 0x09000000 < 0x0A000000 → true
2091–2092	else if ((Address >= (MRAM_END_ADDRESS_CODE - halfSize - MRAM_IP_DATA_IN_CODE_MRAM_SIZE_U32)) && (Address < (MRAM_END_ADDRESS_CODE - halfSize)))	0x09000000 >= 0x08FFD800 → true and 0x09000000 < 0x09000000 → false
2103	return otaAddress;	returns 0x09000000

Result

The MRAMC programs the page at WADDR = 0x09000000 (logical). Logical pair 4 is backed by physical pair 0 (lower bank).

Physical array, pair 0 offset 0x000000: A0 A1 A2 A3 A4 A5 A6 A7 A8 A9 AA AB AC AD AE AF B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 BA BB BC BD BE BF

B3 Boot after the next update (OTA_SEL = 0): MRam_Ip_Init latches the OTA configuration

State of the device

GPR_RWD_0 = 0x00000000 (OTA_SEL = 0, OTA_NUM_FIXED_LPART = 0x00): no swap. Logical pair → physical pair: 0→0, 1→1, 2→2, 3→3, 4→4, 5→5, 6→6, 7→7.

After the next update the device resets without swap. MRam_Ip_Init latches "no swap" again.

Path through the code (main calls)

```

Mem_43_MRAM_Init() Mem_43_MRAM.c:1286
MRam_Ip_Init() MRam_Ip.c:5474
MRam_Ip_GetOtaNumFixed() → 0x10 MRam_Ip.c:2028

```

MRam_Ip_GetOtaNumFixed reads GPR_RWD_0 once, at init

Line	Code executed	Result (values from the run)
2031	uint32 otaNumFixed = MRAM_IP_OTA_NUM_FIXED_ALL;	otaNumFixed = 0x10
2035	uint32 gprVal = IP_GPR_1->GPR_RWD_0;	gprVal = 0
2037	if ((gprVal & K566_GPR1_GPR_RWD_0_OTA_SEL_MASK) != 0U)	0 != 0 → false
2045	return otaNumFixed;	returns 0x10

Result

MRam_Ip_Init (line 5490) keeps the returned value: MRam_Ip_OtaNumFixed = 0x10 (no swap). Every Read / Write / Erase until the next reset uses this value; GPR_RWD_0 is not read again.

R4 Swapped back (next update, OTA_SEL = 0): Fee reads 32 bytes from Cluster0 at 0x08FFD820
State of the device

GPR_RWD_0 = 0x00000000 (OTA_SEL = 0, OTA_NUM_FIXED_LPART = 0x00): **no swap**. Logical pair → physical pair: 0→0, 1→1, 2→2, 3→3, 4→4, 5→5, 6→6, 7→7.

Fee reads the record written in W3 while the swap was active. No swap now: 0x08FFD820 is physical pair 3, where W3 wrote it, so Fee gets C0..DF.

Path through the code (main calls)

```
Mem_43_MRAM_Read(SourceAddress=0x08FFD820, Length=0x20) → 0 Mem_43_MRAM.c:1555
Mem_43_MRAM_GetSectorBatch(Address=0x08FFD820) Mem_43_MRAM.c:443 // MemSectorBatch FeeCluster0 [0x08FFD800,
0x08FFFFFF]
MRam_Ip_Read(SrcAddress=0x08FFD820, SizeInBytes=0x20) → MRAM_IP_STATUS_SUCCESS MRam_Ip.c:7511
MRam_Ip_GetRebasedAddress(Address=0x08FFD820) → 0x08FFD820 MRam_Ip.c:2059
MRam_Ip_ReadData(ReadAddress=0x08FFD820, SizeLeft=0x20) → 4 MRam_Ip.c:1872
... loop: MRam_Ip_ReadData 0x08FFD824, 0x08FFD828, 0x08FFD82C, 0x08FFD830, 0x08FFD834, 0x08FFD838, 0x08FFD83C (4
bytes each)
```

MRam_Ip_GetRebasedAddress which address is really accessed

Line	Code executed	Result (values from the run)
2062	MRam_Ip_AddressType otaAddress = Address;	otaAddress = 0x08FFD820
2064	uint32 halfSize = 0U;	halfSize = 0
2067	if (MRAM_IP_OTA_NUM_FIXED_FULL == MRam_Ip_OtaNumFixed)	0 == 0x10 → false
2072	else if (MRAM_IP_OTA_NUM_FIXED_PARTIAL == MRam_Ip_OtaNumFixed)	4 == 0x10 → false
2082	if (0U != halfSize)	0 != 0 → false
2103	return otaAddress;	returns 0x08FFD820

Result

The CPU reads logical 0x08FFD820 ... → logical pair 3 → **physical pair 3**.

Data returned to the caller (0x08FFD820): C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF D0 D1 D2 D3 D4 D5 D6 D7 D8 D9 DA DB DC DD DE DF

4. Partial swap

PB1 Partial swap product, boot with OTA_SEL = 0: MRam_Ip_Init

State of the device

GPR_RWD_0 = 0x00000100 (OTA_SEL = 0, OTA_NUM_FIXED_LPART = 0x04): **no swap**. Logical pair → physical pair: 0→0, 1→1, 2→2, 3→3, 4→4, 5→5, 6→6, 7→7.

Partial swap product (pairs 0–1 fixed, halves of 12 MB). Boot without swap.

Path through the code (main calls)

```
MRam_Ip_Init() MRam_Ip.c:5474
  MRam_Ip_GetOtaNumFixed() → 0x10 MRam_Ip.c:2028
```

MRam_Ip_GetOtaNumFixed reads GPR_RWD_0 once, at init

Line	Code executed	Result (values from the run)
2031	uint32 otaNumFixed = MRAM_IP_OTA_NUM_FIXED_ALL;	otaNumFixed = 0x10
2035	uint32 gprVal = IP_GPR_1->GPR_RWD_0;	gprVal = 0x100
2037	if ((gprVal & K566_GPR1_GPR_RWD_0_OTA_SEL_MASK) != 0U)	0 != 0 → false
2045	return otaNumFixed;	returns 0x10

Result

MRam_Ip_Init (line 5490) keeps the returned value: MRam_Ip_OtaNumFixed = 0x10 (no swap). Every Read / Write / Erase until the next reset uses this value; GPR_RWD_0 is not read again.

P1 Partial swap product, before the swap: write 32 bytes (D0..EF) at 0x093FD800 (IP level)

State of the device

GPR_RWD_0 = 0x00000100 (OTA_SEL = 0, OTA_NUM_FIXED_LPART = 0x04): **no swap**. Logical pair → physical pair: 0→0, 1→1, 2→2, 3→3, 4→4, 5→5, 6→6, 7→7.

The section of the active half of the Partial swap ends at 0x09400000: [0x093FD800, 0x09400000). A record (D0..EF) is written there without swap: physical pair 4.

Path through the code (main calls)

```
MRam_Ip_Write(DestAddress=0x093FD800, SizeInBytes=0x20) → MRAM_IP_STATUS_SUCCESS MRam_Ip.c:7348
  MRam_Ip_GetRebasedAddress(Address=0x093FD800) → 0x093FD800 MRam_Ip.c:2059
    MRam_Ip_ProcessOperation(DestAddress=0x093FD800, SizeInBytes=0x20) → MRAM_IP_STATUS_SUCCESS MRam_Ip.c:7033
      MRam_Ip_ConfigureLocksInRange(StartAddress=0x093FD800, SizeInBytes=0x20, SetLock=0) → MRAM_IP_STATUS_SUCCESS
MRam_Ip.c:3439
      MRam_Ip_ClearLock(Address=0x093FD800) → MRAM_IP_STATUS_SUCCESS MRam_Ip.c:4432
      MRam_Ip_GetLock(Address=0x093FD800) → MRAM_IP_STATUS_PARTITION_UNLOCKED MRam_Ip.c:4302
      MRam_Ip_ExecuteSinglePage(Address=0x093FD800, WriteSize=0x20) MRam_Ip.c:685
        < MRAMC programs WADDR = 0x093FD800 (logical) → logical pair 4 → physical pair 4, lower bank
      MRam_Ip_ConfigureLocksInRange(StartAddress=0x093FD800, SizeInBytes=0x20, SetLock=1) → MRAM_IP_STATUS_SUCCESS
MRam_Ip.c:3439
      MRam_Ip_SetPartitionLock(Partition=MRAM_IP_PARTITION_LARGE_4_LOWER) MRam_Ip.c:2815
```

MRam_Ip_GetRebasedAddress which address is really accessed		
Line	Code executed	Result (values from the run)
2062	MRam_Ip_AddressType otaAddress = Address;	otaAddress = 0x093FD800
2064	uint32 halfSize = 0U;	halfSize = 0
2067	if (MRAM_IP_OTA_NUM_FIXED_FULL == MRam_Ip_OtaNumFixed)	0 == 0x10 → false
2072	else if (MRAM_IP_OTA_NUM_FIXED_PARTIAL == MRam_Ip_OtaNumFixed)	4 == 0x10 → false
2082	if (0U != halfSize)	0 != 0 → false
2103	return otaAddress;	returns 0x093FD800

Result

The MRAMC programs the page at WADDR = 0x093FD800 (logical). Logical pair 4 is backed by **physical pair 4** (lower bank).

Physical array, pair 4 offset 0x3FD800: D0 D1 D2 D3 D4 D5 D6 D7 D8 D9 DA DB DC DD DE DF E0 E1 E2 E3 E4 E5 E6 E7 E8 E9 EA EB EC ED EE EF

PB2 Partial swap product, boot with OTA_SEL = 1: MRam_Ip_Init**State of the device**

GPR_RWD_0 = 0x00000120 (OTA_SEL = 1, OTA_NUM_FIXED_LPART = 0x04): **Partial swap**. Logical pair → physical pair: 0→0, 1→1, 2→5, 3→6, 4→7, 5→2, 6→3, 7→4.

Boot with the Partial swap selected: MRam_Ip_Init latches 0x04. Logical pairs 2..4 ↔ 5..7, pairs 0..1 fixed.

Path through the code (main calls)

```
MRam_Ip_Init() MRam_Ip.c:5474
  MRam_Ip_GetOtaNumFixed() → 4 MRam_Ip.c:2028
```

MRam_Ip_GetOtaNumFixed reads GPR_RWD_0 once, at init		
Line	Code executed	Result (values from the run)
2031	uint32 otaNumFixed = MRAM_IP_OTA_NUM_FIXED_ALL;	otaNumFixed = 0x10
2035	uint32 gprVal = IP_GPR_1->GPR_RWD_0;	gprVal = 0x120
2037	if ((gprVal & K566_GPR1_GPR_RWD_0_OTA_SEL_MASK) != 0U)	0x20 != 0 → true
2041–2042	otaNumFixed = (gprVal & K566_GPR1_GPR_RWD_0_OTA_NUM_FIXED_LPART_MASK) >> K566_GPR1_GPR_RWD_0_OTA_NUM_FIXED_LPART_SHIFT;	otaNumFixed = 4
2045	return otaNumFixed;	returns 4

Result

MRam_Ip_Init (line 5490) keeps the returned value: MRam_Ip_OtaNumFixed = 0x04 (Partial swap). Every Read / Write / Erase until the next reset uses this value; GPR_RWD_0 is not read again.

P2 Partial swap active: read 32 bytes at 0x093FD800 (IP level)**State of the device**

GPR_RWD_0 = 0x00000120 (OTA_SEL = 1, OTA_NUM_FIXED_LPART = 0x04): **Partial swap**. Logical pair → physical pair: 0→0, 1→1, 2→5, 3→6, 4→7, 5→2, 6→3, 7→4.

Reading 0x093FD800 now: the driver moves it 12 MB higher, to 0x09FFD800, which is physical pair 4 in the Partial swap. The data written in P1 is found again.

Path through the code (main calls)

```
MRam_Ip_Read(SrcAddress=0x093FD800, SizeInBytes=0x20) → MRAM_IP_STATUS_SUCCESS MRam_Ip.c:7511
  MRam_Ip_GetRebasedAddress(Address=0x093FD800) → 0x09FFD800 MRam_Ip.c:2059
    MRam_Ip_ReadData(ReadAddress=0x09FFD800, SizeLeft=0x20) → 4 MRam_Ip.c:1872
      ... loop: MRam_Ip_ReadData 0x09FFD804, 0x09FFD808, 0x09FFD80C, 0x09FFD810, 0x09FFD814, 0x09FFD818, 0x09FFD81C (4
      bytes each)
```

MRam_Ip_GetRebasedAddress which address is really accessed

Line	Code executed	Result (values from the run)
2062	MRam_Ip_AddressType otaAddress = Address;	otaAddress = 0x093FD800
2064	uint32 halfSize = 0U;	halfSize = 0
2067	if (MRAM_IP_OTA_NUM_FIXED_FULL == MRam_Ip_OtaNumFixed)	0 == 4 → false
2072	else if (MRAM_IP_OTA_NUM_FIXED_PARTIAL == MRam_Ip_OtaNumFixed)	4 == 4 → true
2075	halfSize = MRAM_IP_OTA_PARTIAL_SWAP_OFFSET;	halfSize = 0x00C00000
2082	if (0U != halfSize)	0 != 0x00C00000 → true
2084–2085	if ((Address >= (MRAM_END_ADDRESS_CODE - MRAM_IP_DATA_IN_CODE_MRAM_SIZE_U32)) && (Address < MRAM_END_ADDRESS_CODE))	0x093FD800 >= 0x09FFD800 → false and 0x093FD800 < 0x0A000000 → true
2091–2092	else if ((Address >= (MRAM_END_ADDRESS_CODE - halfSize - MRAM_IP_DATA_IN_CODE_MRAM_SIZE_U32)) && (Address < (MRAM_END_ADDRESS_CODE - halfSize)))	0x093FD800 >= 0x093FD800 → true and 0x093FD800 < 0x09400000 → true
2095	otaAddress = Address + halfSize;	otaAddress = 0x09FFD800
2103	return otaAddress;	returns 0x09FFD800

Result

The CPU reads logical 0x09FFD800 ... → logical pair 7 → **physical pair 4**.

Data returned to the caller (0x093FD800): D0 D1 D2 D3 D4 D5 D6 D7 D8 D9 DA DB DC DD DE DF E0 E1 E2 E3 E4 E5 E6 E7 E8 E9 EA EB EC ED EE EF

5. Summary

Step	Swap	Fee / updater asks	Driver accesses	Logical → physical pair	Data
W1	no swap	0x08FFD800	0x08FFD800	3 → 3	A0 A1 A2 A3 ...
W2	no swap	0x09FFD800	0x09FFD800	7 → 7	B0 B1 B2 B3 ...
R1	no swap	0x08FFD800	0x08FFD800	3 → 3	A0 A1 A2 A3 ...
R2	Full	0x08FFD800	0x09FFD800	7 → 3	A0 A1 A2 A3 ...
R3	Full	0x09FFD800	0x08FFD800	3 → 7	B0 B1 B2 B3 ...
W3	Full	0x08FFD820	0x09FFD820	7 → 3	C0 C1 C2 C3 ...
W4	Full	0x09000000	0x09000000	4 → 0	A0 A1 A2 A3 ...
R4	no swap	0x08FFD820	0x08FFD820	3 → 3	C0 C1 C2 C3 ...
P1	no swap	0x093FD800	0x093FD800	4 → 4	D0 D1 D2 D3 ...
P2	Partial	0x093FD800	0x09FFD800	7 → 4	D0 D1 D2 D3 ...

Cluster0 always lives in physical pair 3 and Cluster1 in physical pair 7, whatever the swap state; the updater writes the passive image where it asks. The only condition for the integrator: the images must not use the last `MemDataInMRamCodeSize` bytes of either half.